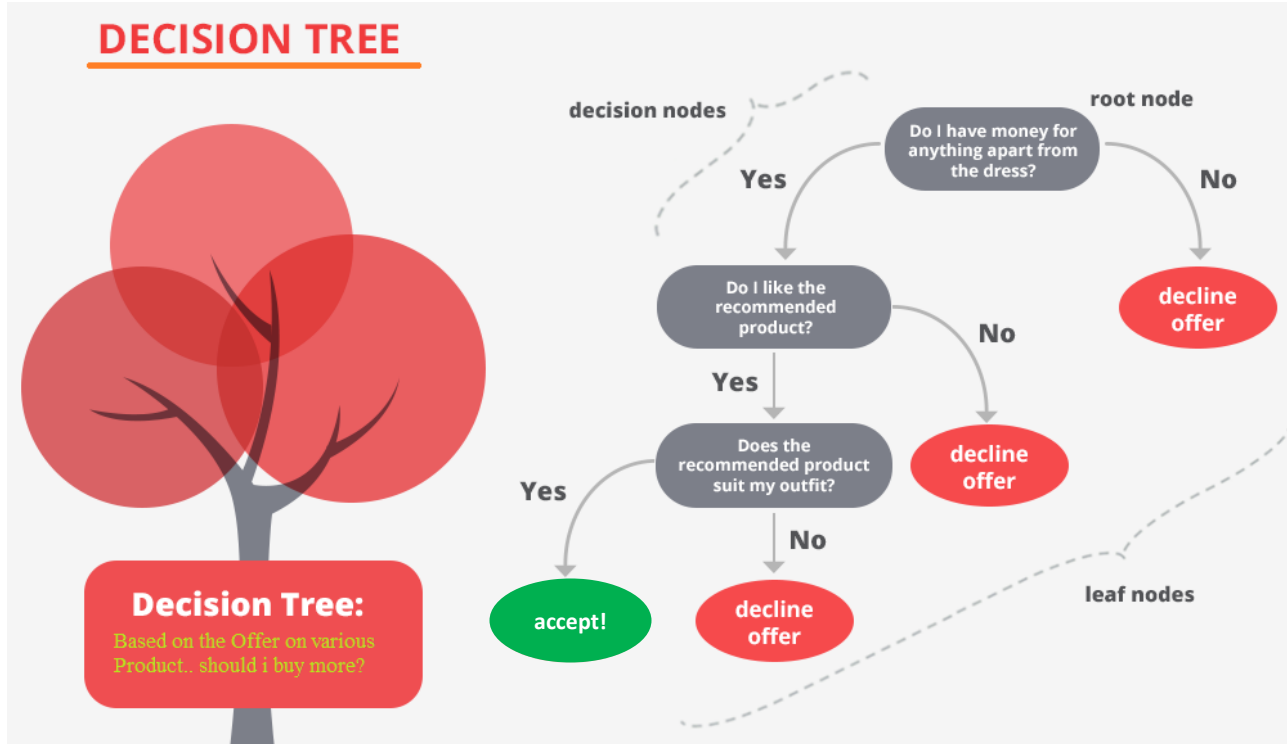


Decision Trees (DT)



Gerry Tsoukalas

Image source: <https://blog.paperspace.com/decision-trees/>

Before we get started...

- Always check syllabus in case of updated schedule
- Meet at least once a week with your project teams

What are Decision Trees?

- A **popular** ML method for prediction, like k-NN and Naïve Bayes, pioneered by Breiman et al. (1984)
 - *Quote: “If one had to choose a classification technique that performs well across a wide range of situations without requiring much effort from the analyst while being readily understandable by the consumer of the analysis, a strong contender would be Decision Trees” (DMBA p. 218)*
- The output is not just a predicted result, ‘y=0’ or ‘y=1’, or a probability, $p(y=1|x)$, but also a set of general “rules” for **why** we arrived at the result:
 - **Remember for k-NN:**
 - Collect $x=\{\text{income}=65,000, \text{family members} = 1, \text{credit card balance} = 1000\}$, then find neighbors, and determine $y:\{\text{“loan default”}\}$ class (but we don’t know anything more than that)
 - **But Trees are different:** generate rules
 - If $x=\{\text{income}>60,000, \text{family members} < 2, \text{education} < \text{PhD}\}$, then $y=\{\text{“loan accept”}\}=0$ (and we know why)
 - If $x=\{\text{income}<60,000, \text{family members} < 2, \text{education} > \text{BS}\}$, then $y=\{\text{“loan accept”}\}=1$ (and we know why)
 - ...

What are Decision Trees?

Also called **Classification** and **Regression** Trees (CART)

- **Classification** Trees

- When the output represents a class, like $y = \text{'spam'}$ or $y = \text{'ham'}$, or $y = 0$ or $y = 1$

- **Regression** Trees

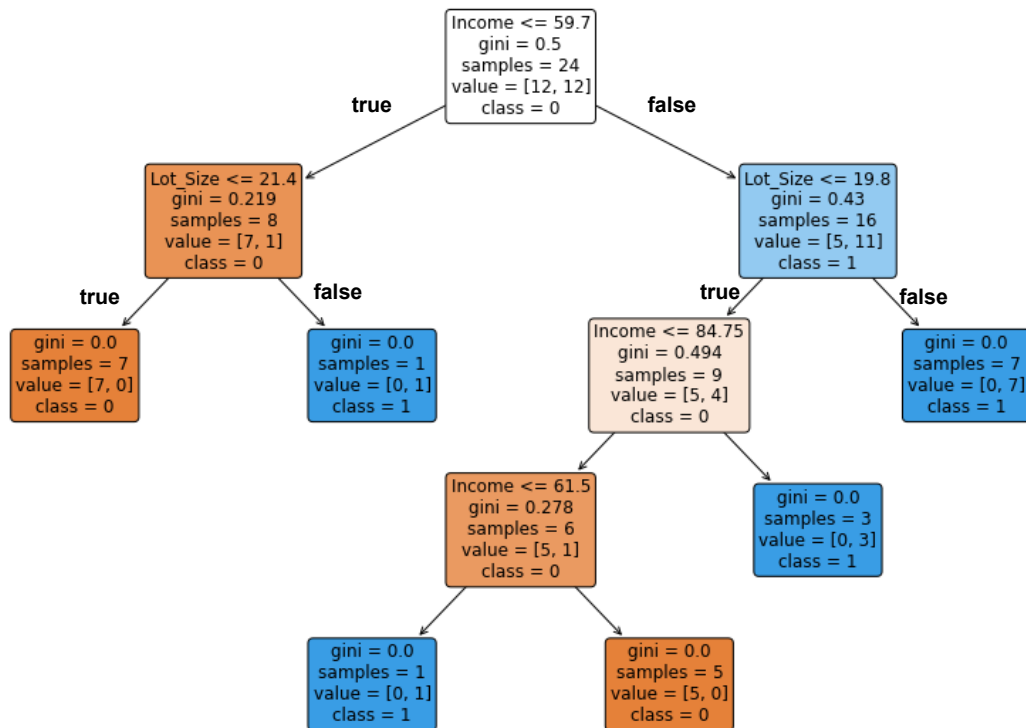
- When the output represents a continuous value, e.g., predicting house prices in a given area

- In both cases, **the inputs can be a mixture of categorical and numerical**

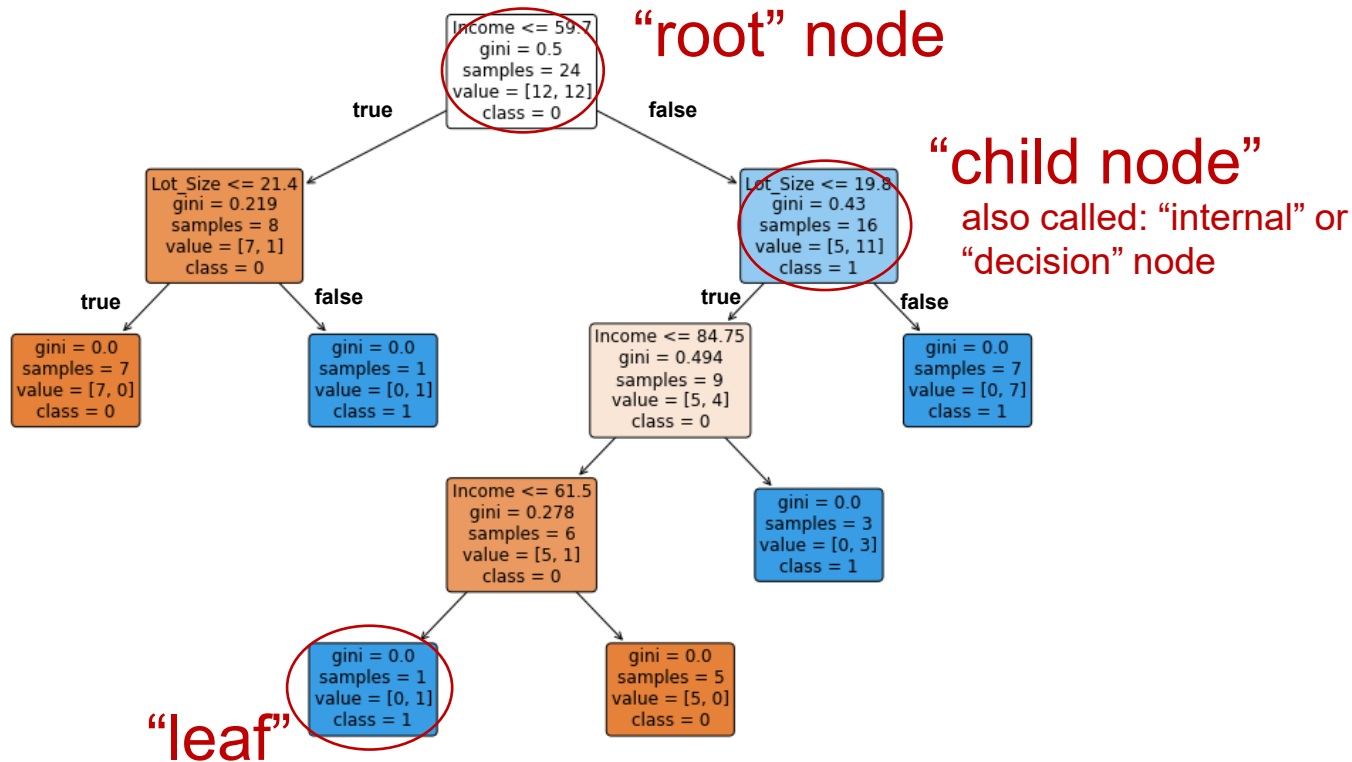
- k-NN works best with numerical inputs but can accommodate some categorical as well
 - Multinomial Naïve Bayes works best with categorical inputs
 - Gaussian Naïve Bayes (which we did not see) works best with numerical inputs
 - CART is more flexible!

Classification Trees

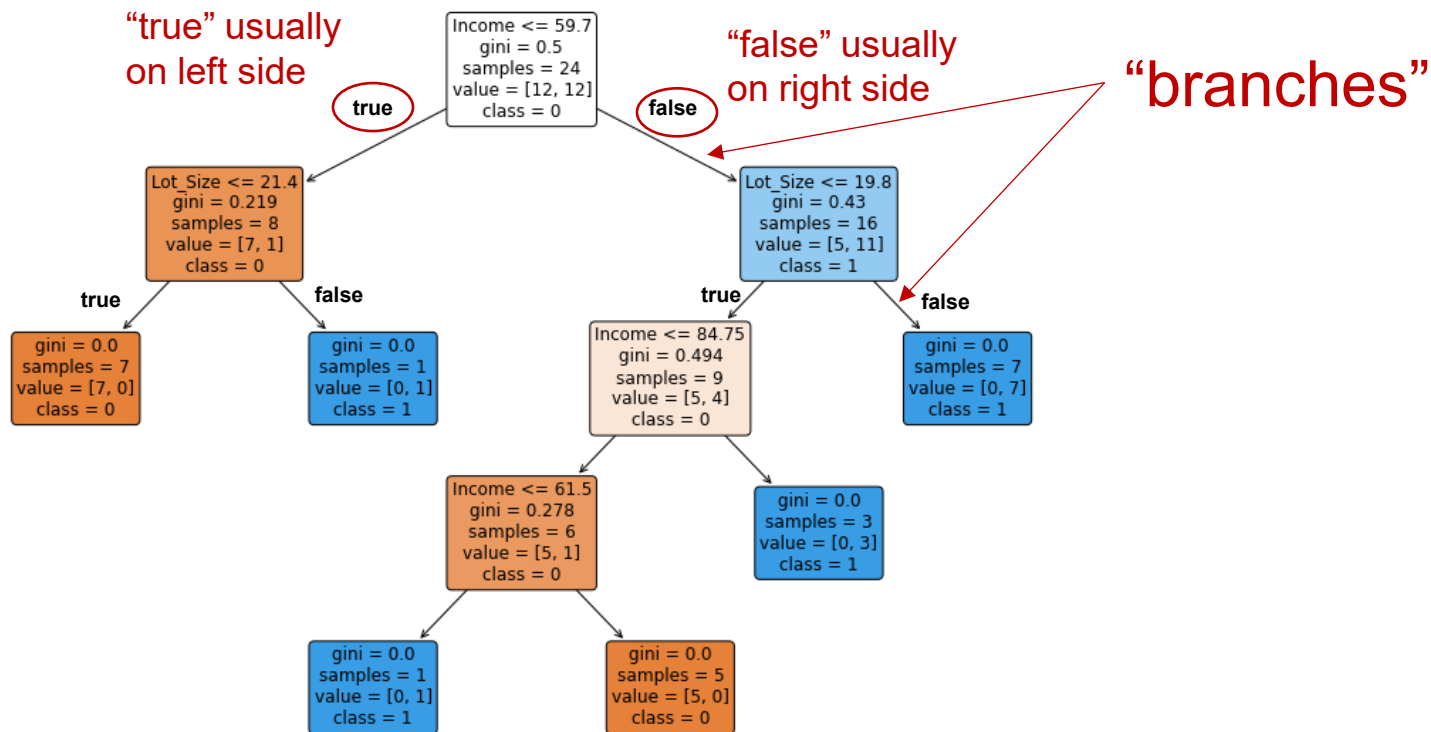
Classification Trees – Rules Visualized / Vocabulary



Classification Trees – Rules Visualized / Vocabulary



Classification Trees – Rules Visualized / Vocabulary

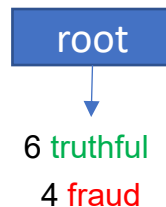


Classification Trees – Build from the data...

What's the $p(\text{fraud}|x)$?

<i>x</i>		<i>y</i>
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

Step 1: look at the root (meaning, all data) and take majority vote.



Majority vote: "truthful"

Step 2: what's the error rate if we classify all rows as truthful? (note, other criteria could have been chosen)

6 truthful correctly identified

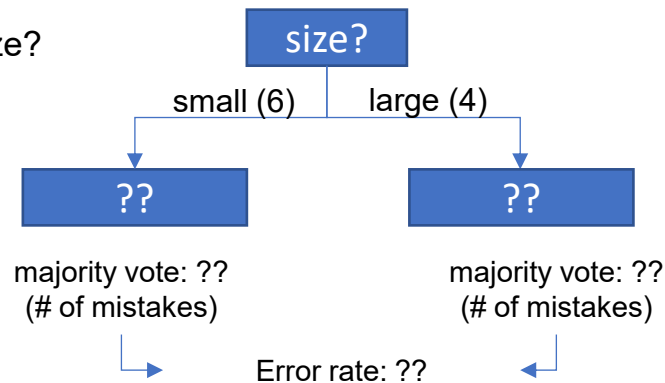
4 fraud incorrectly identified as truthful

Error rate: $4/(4+6) = 40\%$

Classification Trees – Build from the data...

X		y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

Try size?

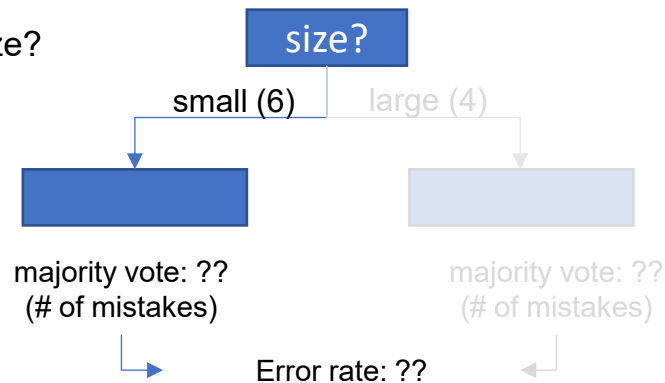


Step 3: try to split via each column separately

Classification Trees – Build from the data...

X		y	
Charges?	Size	Outcome	
yes	small	truthful	#1
no	small	truthful	#2
no	large	truthful	
no	large	truthful	
no	small	truthful	#3
no	small	truthful	#4
yes	small	fraud	#1
yes	small	fraud	#2
no	large	fraud	
yes	large	fraud	

Try size?



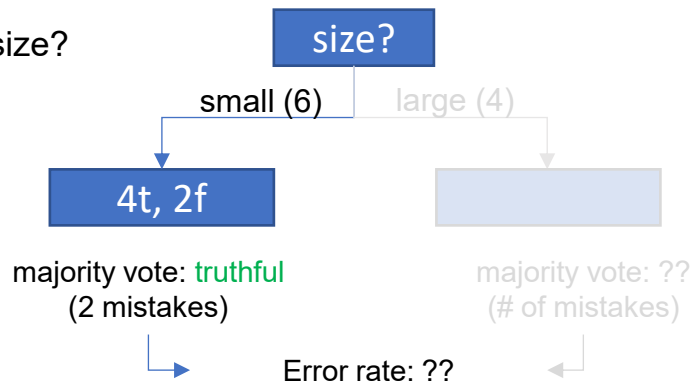
Step 3: try to split via each column separately

Classification Trees – Build from the data...

X Y

Charges?	Size	Outcome	
yes	small	truthful	#1
no	small	truthful	#2
no	large	truthful	
no	large	truthful	
no	small	truthful	#3
no	small	truthful	#4
yes	small	fraud	#1
yes	small	fraud	#2
no	large	fraud	
yes	large	fraud	

Try size?



Step 3: try to split via each column separately

Classification Trees – Build from the data...

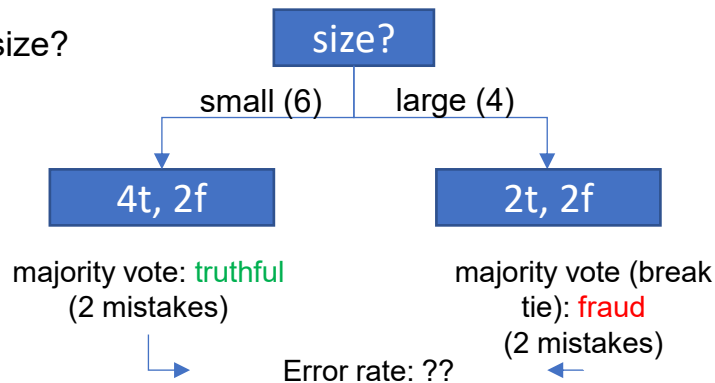
X

Charges?	Size
yes	small
no	small
no	large
no	large
no	small
no	small
yes	small
yes	small
no	large
yes	large

y

Outcome
truthful
truthful
truthful
truthful
truthful
truthful
fraud
fraud
fraud
fraud

Try size?

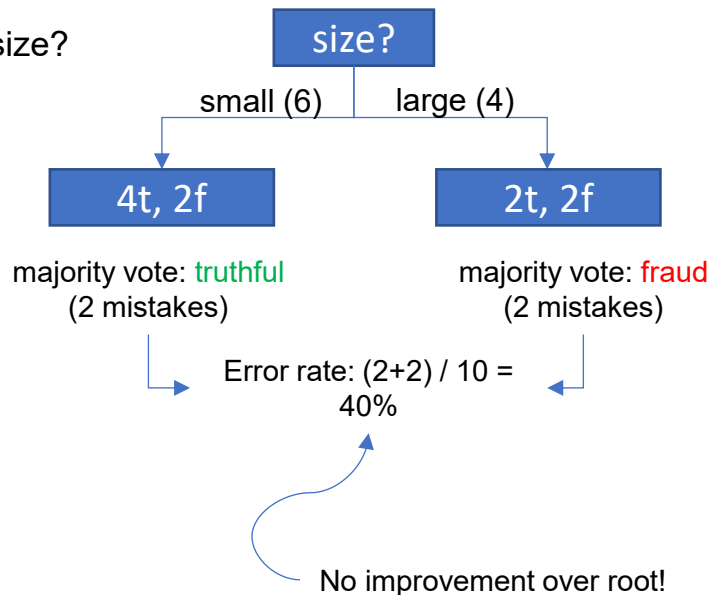


Step 3: try to split via each column separately

Classification Trees – Build from the data...

X		Y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

Try size?

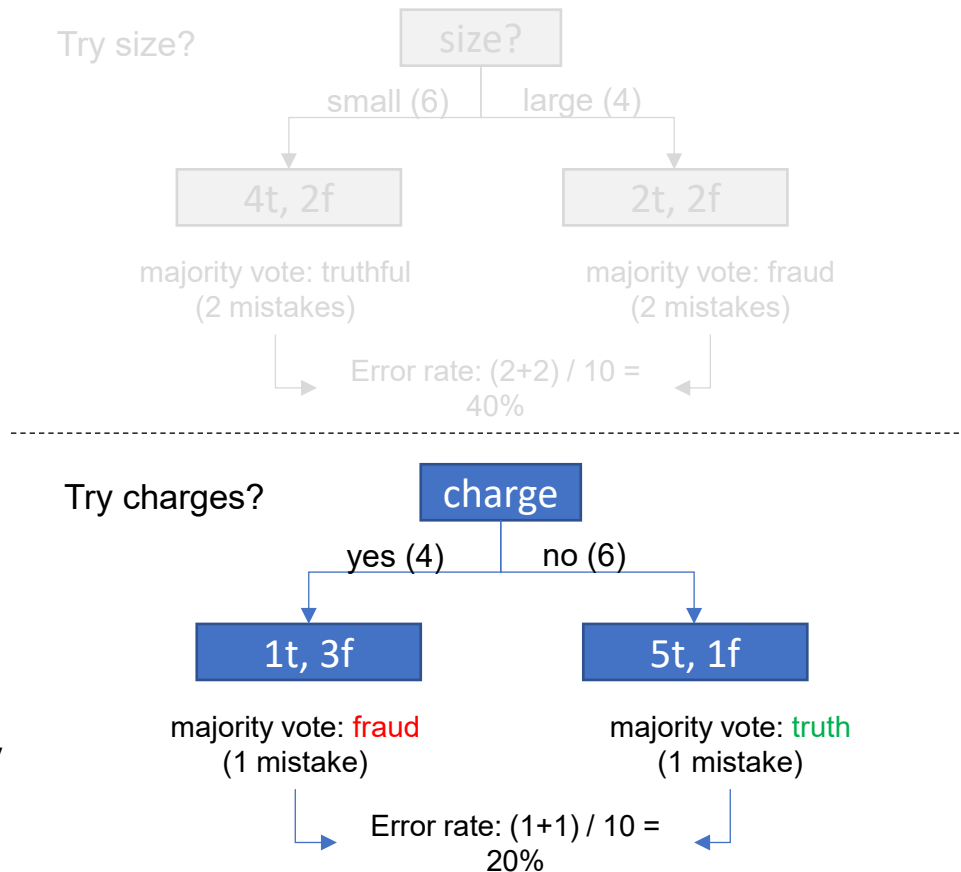


Step 3: try to split via each column separately

Classification Trees – Build from the data...

X		y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

Step 3: try to split via each column separately



Classification Trees – Which split is best?

Split decision	Error Rate
Root	40%
Split on “size”	40%
Split on “charges”	20%

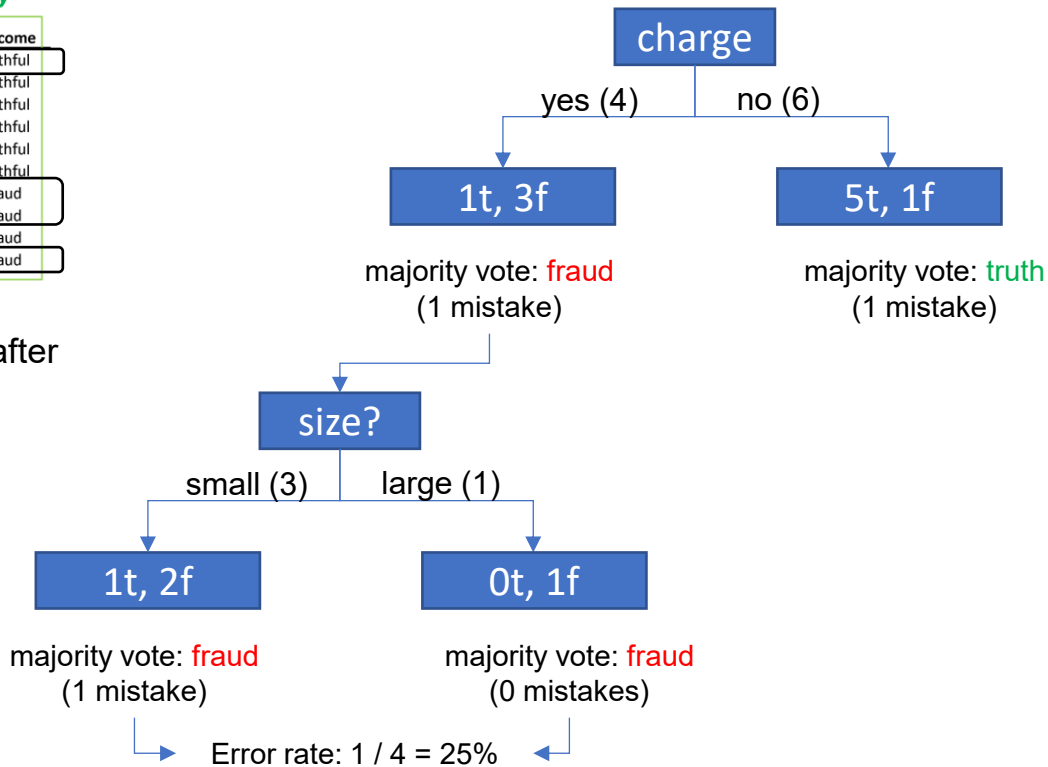
Conclusion

- split on charges! It has lower error rate
- But wait... why stop here? We can continue splitting!

Classification Trees – Build from the data...

x		y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

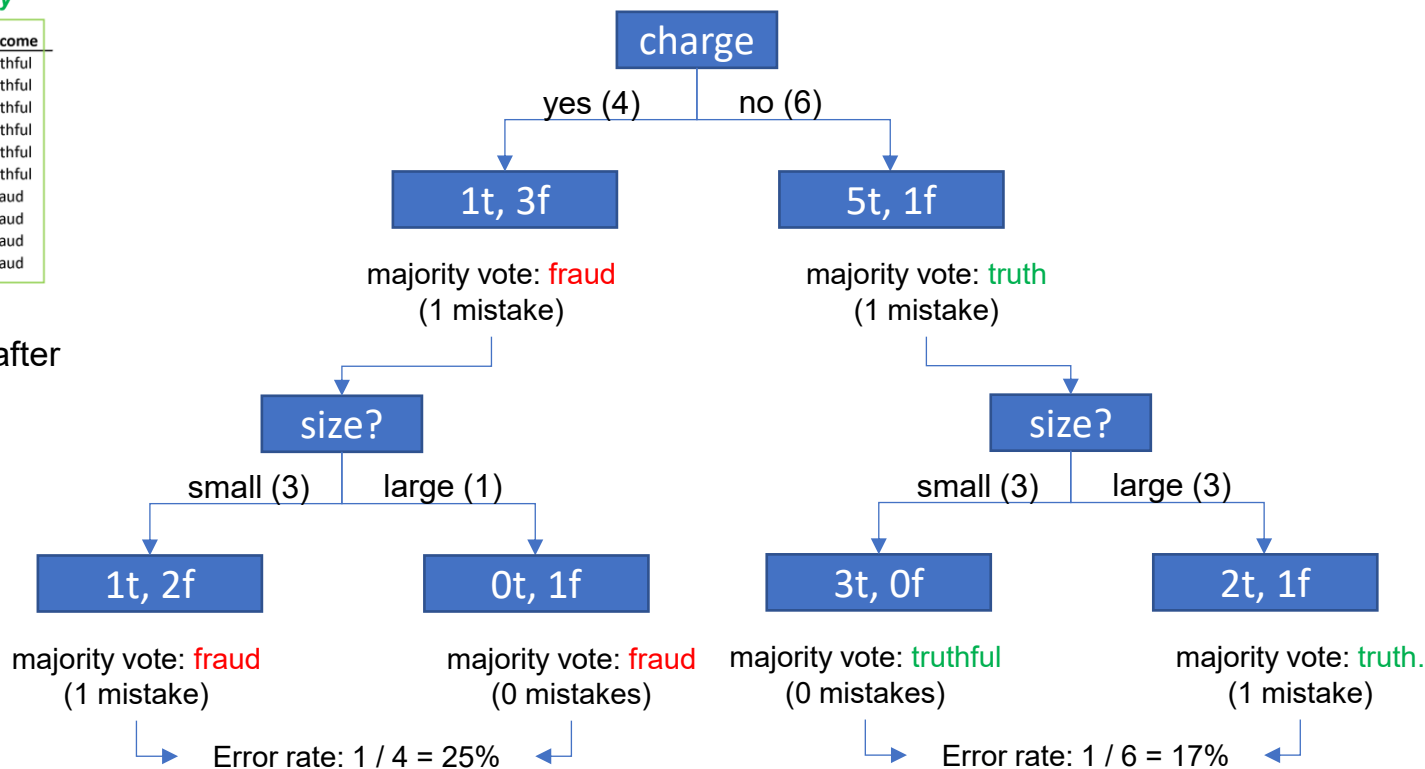
Step 4: split on size after split on charges



Classification Trees – Build from the data...

x		y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes	small	fraud
yes	small	fraud
no	large	fraud
yes	large	fraud

Step 4: split on size after split on charges



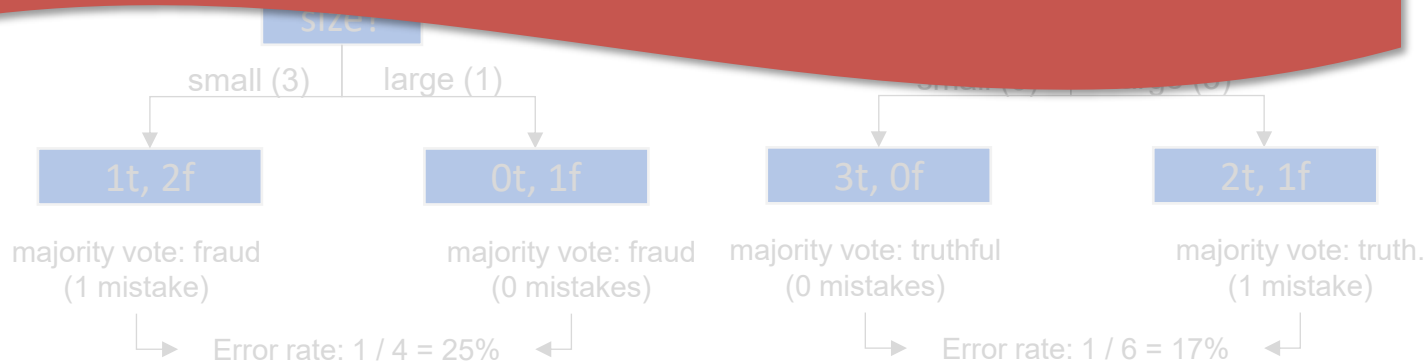
Classification Trees – Build from the data...

X		Y
Charges?	Size	Outcome
yes	small	truthful
no	small	truthful
no	large	truthful
no	large	truthful
no	small	truthful
no	small	truthful
yes		
yes		
no		
yes		

no improvement, overall error rate is still 20%

$$\text{Total errors} / \text{total sample} = (1+1)/(4+6)$$

Step 4: split on ch

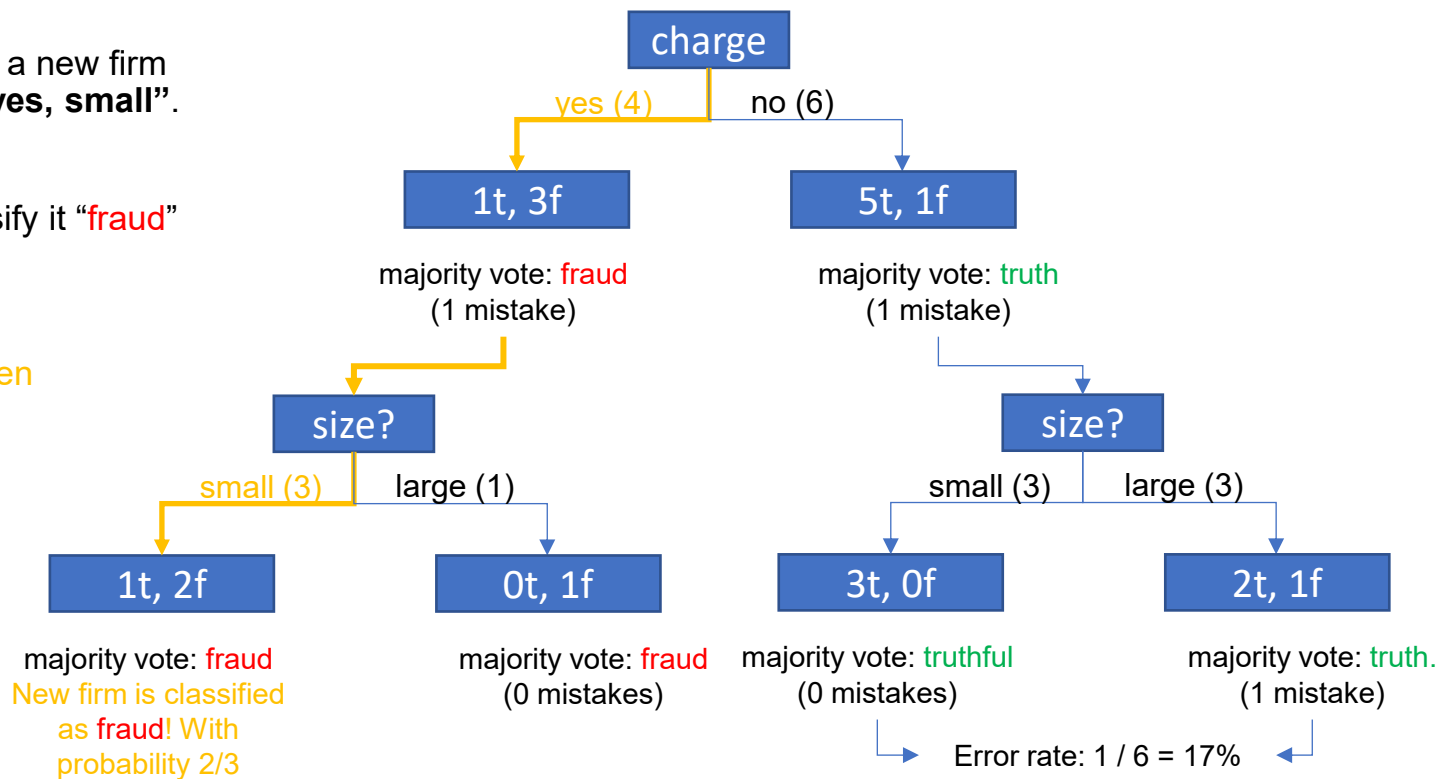


Classification Trees – How to make predictions

Step 5: pretend a new firm arrives that is “yes, small”.

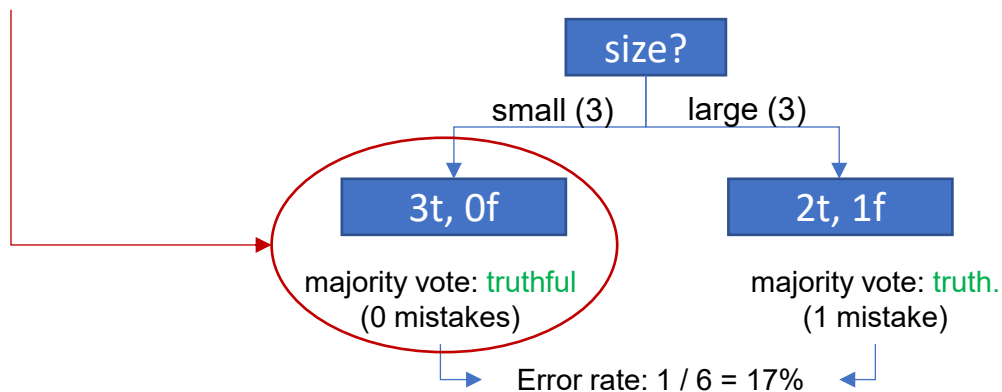
Should we classify it “**fraud**” or “**truthful**”?

..follow the **golden** branches!



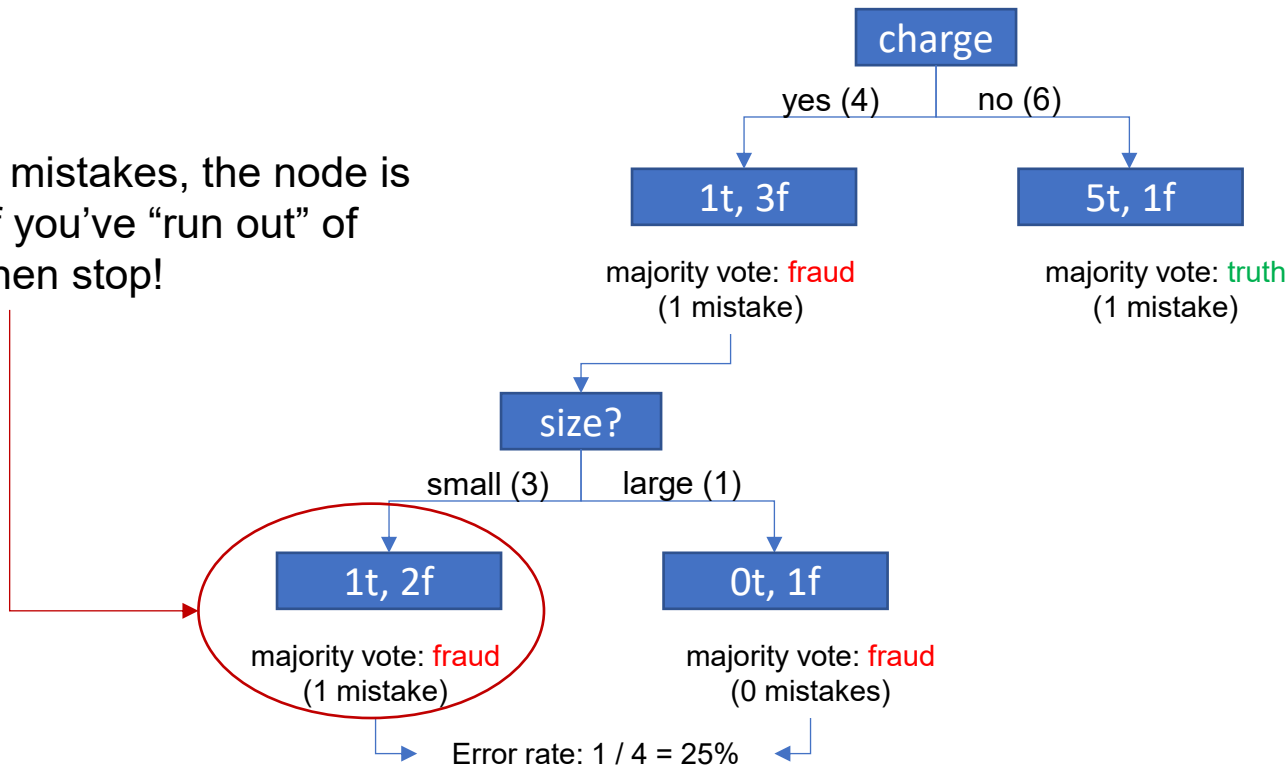
Classification Trees – Stopping Criteria #1

When reaching a node with 0 mistakes, stop!
It's called forming a "pure leaf"



Classification Trees – Stopping Criteria #2

If reaching a node with mistakes, the node is termed “impure”, but if you’ve “run out” of columns to split on... then stop!



Classification Trees – Full Tree

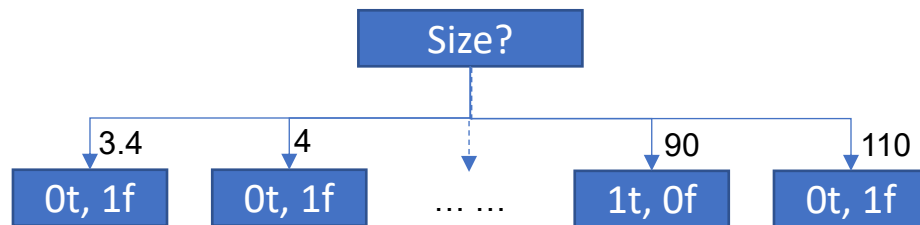
If these are the only 2 stopping criteria you use, you will end up building the so-called “**Full Tree**”...

... usually, the “Full Tree” overfits the data, so we'll have to take extra steps from there... but it's useful to have it.

Classification Trees – What if inputs are numerical?

Imagine size column looked like this:

X		y
Charges?	Size	
yes	10	truthful
no	5	truthful
no	90	truthful
no	80	truthful
no	9.5	truthful
no	8.8	truthful
yes	3.4	fraud
yes	4	fraud
no	95	fraud
yes	110	fraud



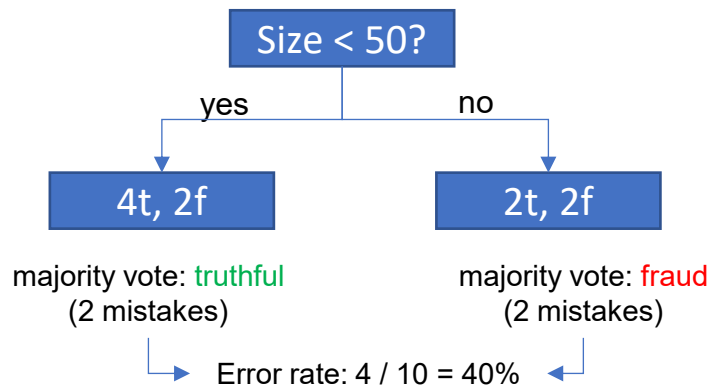
Error rate: $0 / 10 = 0\%$!

1 data point only per cell, **overfitting alert!!!**

Instead, pick a size threshold, e.g., 50!

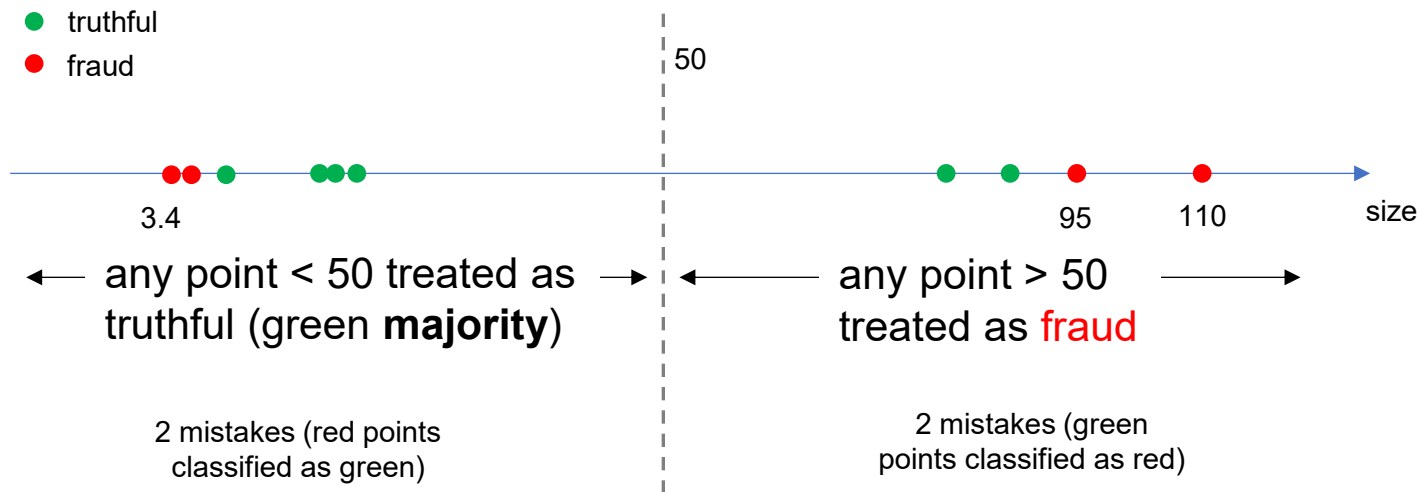
Classification Trees – What if inputs are numerical?

x		y
Charges?	Size	Outcome
yes	10	truthful
no	5	truthful
no	90	truthful
no	80	truthful
no	9.5	truthful
no	8.8	truthful
yes	3.4	fraud
yes	4	fraud
no	95	fraud
yes	110	fraud



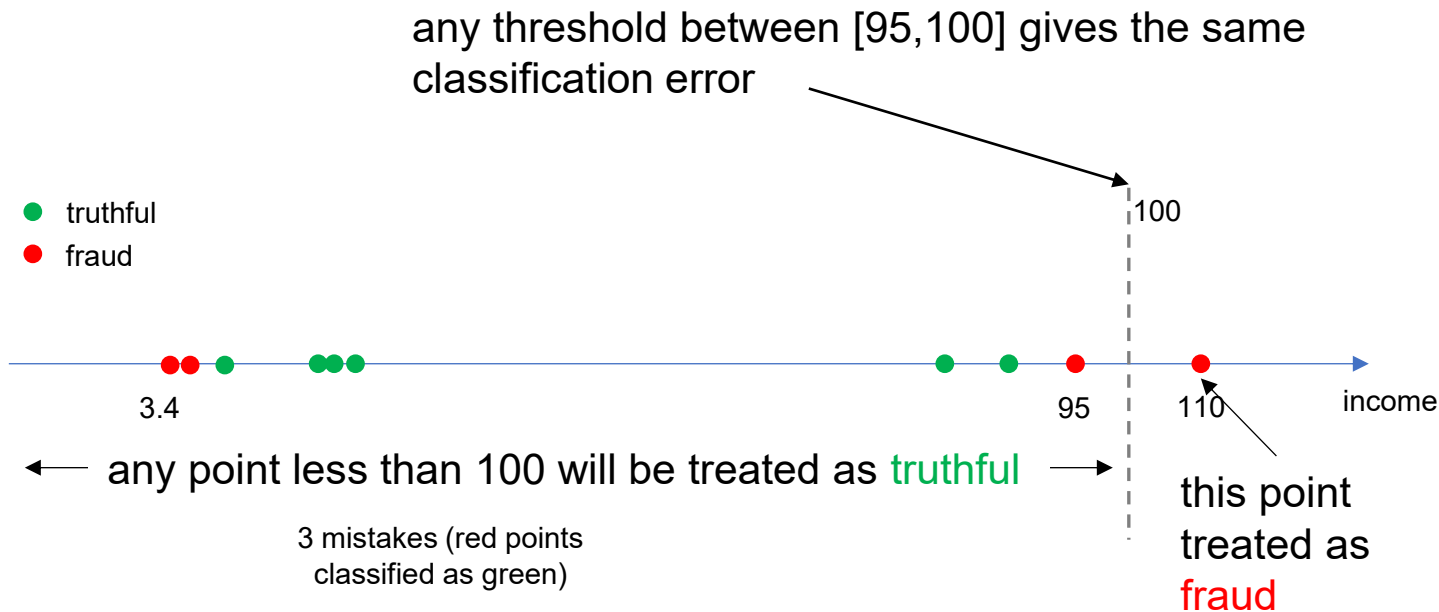
How to pick the threshold?
(is <50 the best one?)

Classification Trees – how to pick threshold?



error rate of “threshold 50” = $4/(6+4) = 40\%$

Classification Trees – let's try threshold=100



error rate of “threshold 100” = $3/(9+1) = 33\%$

... therefore, this might be better than “threshold 50”... rinse and repeat for all others!

Alternative metrics for error measurement

- We used “error rates” but there are two alternative error metrics used for classification trees, that perform “better” than just counting # of mistakes
- **Alternative #1: the “Gini” impurity index**
- Answers the question: If I randomly select an item from a given node, and then guess its class, what’s the chance I’ll be wrong? The guessing follows the class distribution
- **Example:** 8 blue and 2 red balls in a jar.

$$P(\text{pick blue and guess red}) = 8/10 * (1 - 8/10)$$

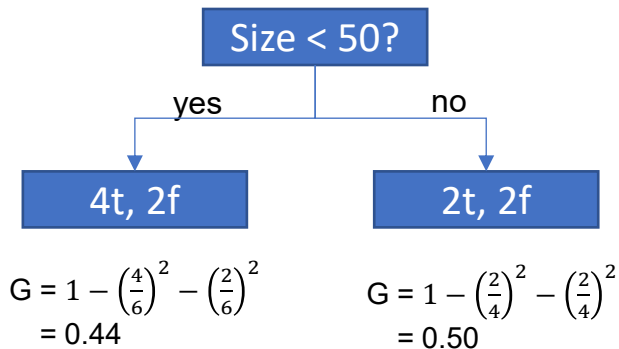
$$P(\text{pick red and guess blue}) = 2/10 * (1 - 2/10)$$

$$P(\text{wrong}) = p_{\text{blue}} * (1 - p_{\text{blue}}) + p_{\text{red}} * (1 - p_{\text{red}}) = 1 - p_{\text{blue}}^2 - p_{\text{red}}^2$$

For m different classes:
$$G = 1 - \sum_{k=1}^m p_k^2 = 1 - p_1^2 - p_2^2 - \dots - p_m^2$$

Alternative metrics for error measurement

$$G = 1 - \sum_{k=1}^m p_k^2 = 1 - p_1^2 - p_2^2 - \dots - p_m^2$$



- if all points are in one class (0 mistakes), the Gini index will be 0. Proof: $1 - 1^2 - 0^2 = 1 - 1 = 0$
- if half the points are misclassified, the Gini index will be 0.5. Proof: $1 - 0.5^2 - 0.5^2 = 1 - 0.25 - 0.25 = 0.5$
- once each box's G index is computed, we average them, weighted by the number of data points coming from each one.
- The left box has 6 points (4t, 2f) and right one has 4 points (2t, 2f), for a total of 10 points across the two boxes. The combined G index for "size < 50" is $G = 0.44 * 6/10 + 0.50 * 4/10 = 0.46$ (in other words, pretty bad)

Alternative metrics for error measurement

- There are two alternative error metrics used for classification: error rate and Gini score.

- **Alternative**

The Gini score does a **better job** handling **imbalanced** datasets, so it's generally preferred over the error rate criterion.

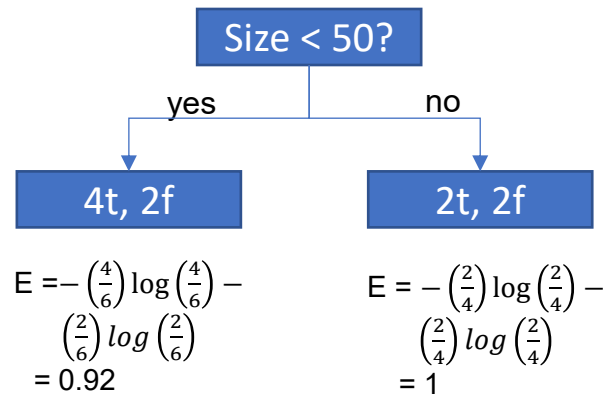
- For a given dataset, the error rate is the proportion of misclassified points.
- if all points are correctly classified, the error rate is 0.
- if half the points are misclassified, the error rate will be 0.5. Proof: $1 - 0.5 - 0.5 = 1 - 0.25 - 0.25 = 0.5$
- once each box's G index is computed, we average them, weighted by the number of data points coming from each one. The left box has 6 points (4t, 2f) and right one has 4 points (2t, 2f), for a total of 10 points across the two boxes. The combined G index for "size <50" is $G = 0.44 * 6/10 + 0.50 * 4/10 = 0.46$ (in other words, pretty bad)

Alternative metrics for error measurement

- **Alternative #2: the “entropy” measure**
- Intuition: if I designed the most efficient yes/no question strategy possible to identify which class an item belongs to, how many questions would I need on average?

$$E = - \sum_{k=1}^m p_k \log_2(p_k) \\ = -p_1 \log_2(p_1) - \dots - p_m \log_2(p_m)$$

- The lower the entropy the better. The best possible entropy score when there are no mistakes is zero.
- The entropy is worst at value $\log_2(m)$ (to be read as “log base 2 of m”), which is equal to 1 when there are just 2 possible classes (yes/no, 0/1, etc).
- The combined entropy of “size <50” box is (like before) the weighted average. Combined $E = 0.92 * 6/10 + 1 * 4/10 = 0.95$



Alternative metrics for error measurement

- **Alternative #2: the “entropy” measure**

Entropy sometimes works better to address imbalanced datasets, but takes longer to compute. Bottom line: try both! (as with many things in data science)

- The score
- The 2 of class
- The combined entropy of “size <50” box is (like before) the weighted average. Combined $E = 0.92 * 6/10 + 1 * 4/10 = 0.95$

Size < 50?

2f

$$\log\left(\frac{2}{4}\right) -$$
$$g\left(\frac{2}{4}\right)$$

Visualizations of Decision Trees

- New example: Classify 24 households as owning or not owning a “product”
- Predictors = Income, Lot Size
- How to split
 - Order records according to one variable, say income
 - Take a predictor value, say 59.7 (the first record) and divide records into those with income ≥ 59.7 and those < 59.7
 - Measure resulting purity (homogeneity) of class in each resulting portion
 - Try all other split values
 - Repeat for other variable(s)
 - Select the one variable & split that yields the most purity
- For categorical variables examine all possible ways of split
 - e.g., categories A, B, C can be split 3 ways
 - {A} and {B, C}
 - {B} and {A, C}
 - {C} and {A, B}
 - With many categories, # of splits becomes huge

Income	Lot Size	Ownership
60.0	18.4	owner
85.5	16.8	owner
64.8	21.6	owner
61.5	20.8	owner
87.0	23.6	owner
110.1	19.2	owner
108.0	17.6	owner
82.8	22.4	owner
69.0	20.0	owner
93.0	20.8	owner
51.0	22.0	owner
81.0	20.0	owner
75.0	19.6	non-owner
52.8	20.8	non-owner
64.8	17.2	non-owner
43.2	20.4	non-owner
84.0	17.6	non-owner
49.2	17.6	non-owner
59.4	16.0	non-owner
66.0	18.4	non-owner
47.4	16.4	non-owner
33.0	18.8	non-owner
51.0	14.0	non-owner
63.0	14.8	non-owner

Visualizations of Decision Trees

- Full (solved) tree, [0 is non-owner, 1 is owner]

- Root:

- [12,12], gini=0.5 (terrible)

- First split Income @ 59.7

- <59.7, yields [7,1], gini=0.219, y_pred=0
 - >59.7, yields [5,11], gini=0.43, y_pred=1

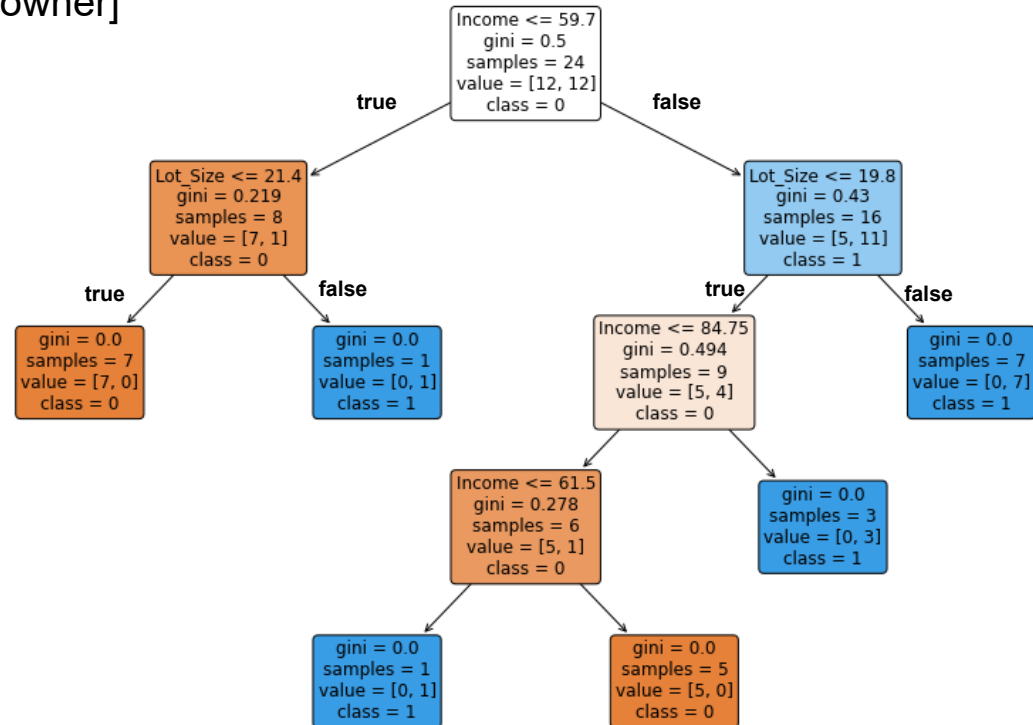
- Second split (left)

- Income < 59.7 AND Lot_Size < 21.4
 - yields [7,0], gini=0 (leaf!), y_pred=0
 - Income < 59.8 AND Lot_Size > 21.4
 - yields [0,1], gini=0 (leaf!), y_pred=1

- Second split (right)

- Income > 59.7 AND Lot_size < 19.8
 - yields [5,4], gini 0.494 (pretty bad), y_pred=0
 - Income > 59.7 AND Lot_size > 19.8
 - ...

- etc...



Visualizations of Decision Trees

- Full (solved) tree, [0 is non-owner, 1 is owner]

- Root:

- [12,12], gini=0.5 (terrible)

- First split Income @ 59.7

- <59.7, yields [7,1], gini=0.219, y_pred=0
 - >59.7, yields [5,11], gini=0.43, y_pred=1

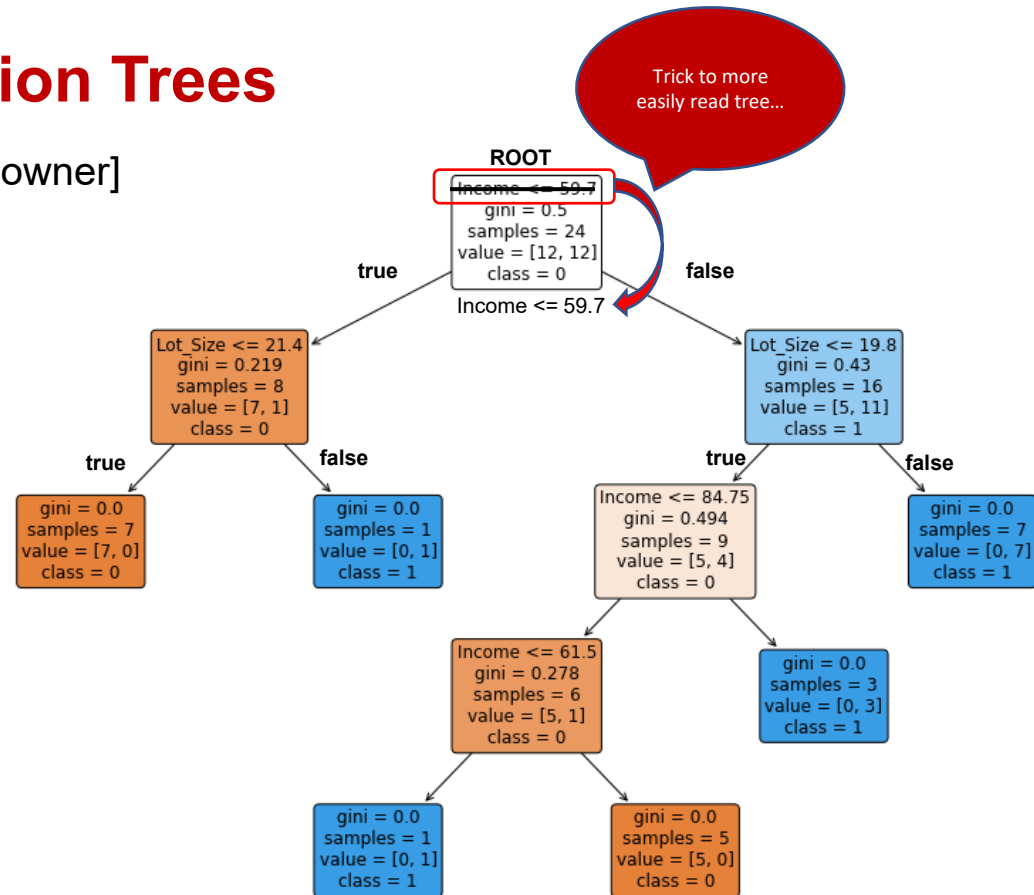
- Second split (left)

- Income<59.7 AND Lot_Size < 21.4
 - yields [7,0], gini=0 (leaf!), y_pred=0
 - Income<59.8 AND Lot_Size > 21.4
 - yields [0,1], gini=0 (leaf!), y_pred=1

- Second split (right)

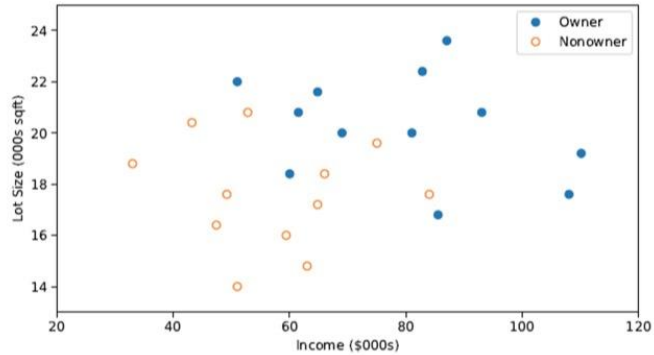
- Income>59.7 AND Lot_size < 19.8
 - yields [5,4], gini 0.494 (pretty bad), y_pred=0
 - Income>59.7 AND Lot_size > 19.8
 - ...

- etc...



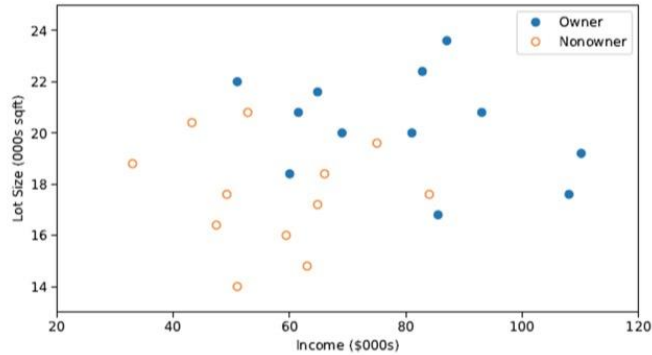
Visualizations of Decision Trees

Raw (unsplit) data

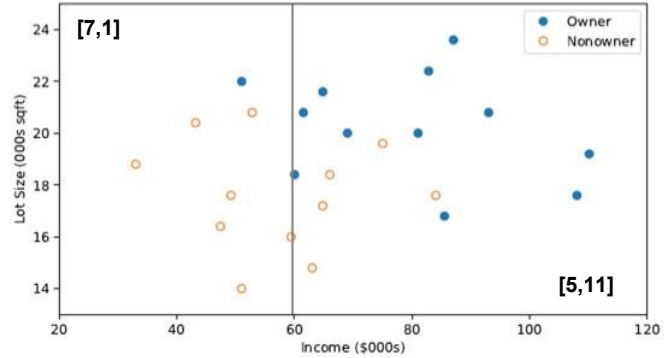


Visualizations of Decision Trees

Raw (unsplit) data

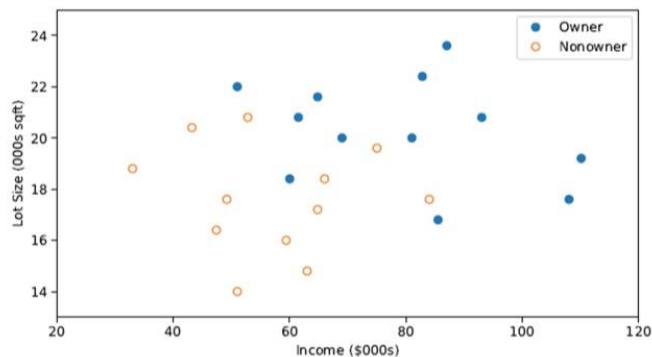


The First Split: Income = 59.7

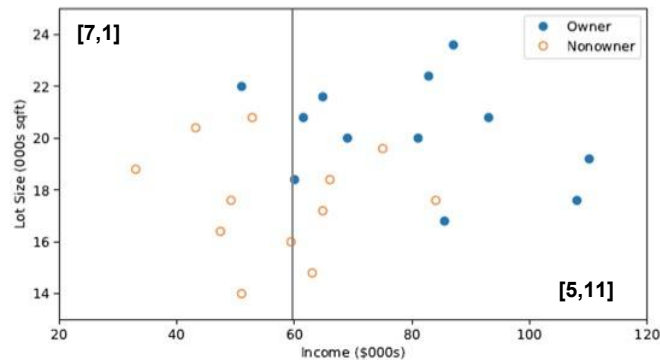


Visualizations of Decision Trees

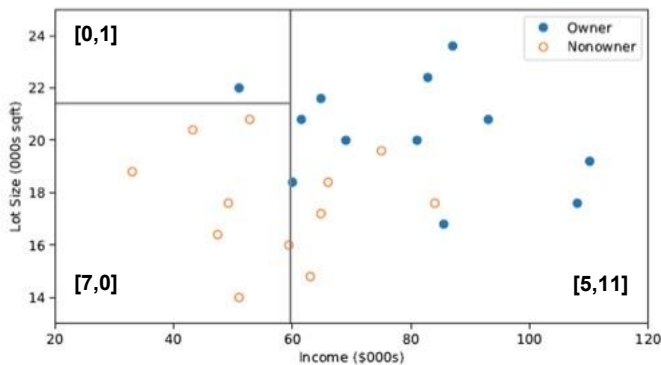
Raw (unsplit) data



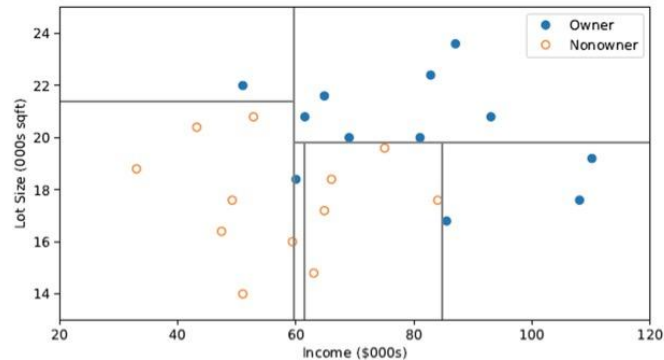
The First Split: Income = 59.7



Second Split: Lot size = 21.4



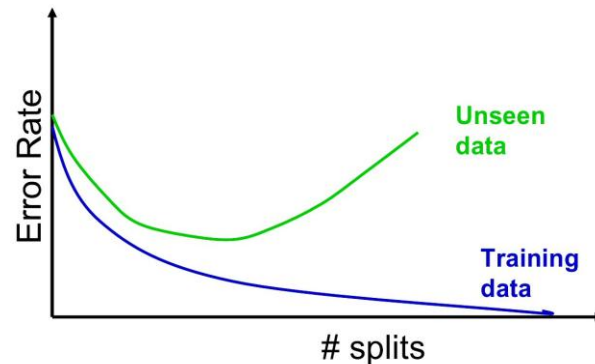
After All Splits (Full Tree)



...

Performance Evaluation and “Overfitting”

- Usual confusion matrices apply!
- The overfitting problem
 - Full trees are complex and overfit the data
 - Natural end of process is 100% purity in each leaf
 - These trees fail when they see new data
- Trees can also be unstable
 - If 2 or more variables are of roughly equal importance, which one CART chooses for the first split can depend on the initial partition into training and validation
 - A different partition into training/validation could lead to a different initial split
 - This can cascade down and produce a very different tree from the first training/validation partition
 - Solution is to try many different training/validation splits – k-fold “**cross validation**” is going to be an important factor for decision trees in general.



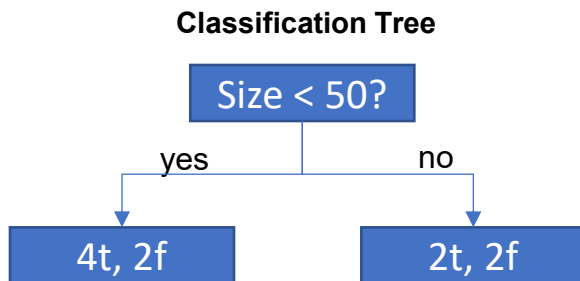
How to limit “tree growth” for optimal accuracy?

- We usually impose additional restrictions such as
 - Maximum tree depth (longest path between the root node and the leaf node).
 - Minimum number of records in split nodes
 - Minimum number of records in terminal node
 - Minimum impurity increase needed to conduct a split on the next column
- But what are optimal values for these parameters?
- Use Grid or Randomized Search on training data, coupled with cross-validation
 - Build multiple trees using different values for the parameters, using the training fold
 - Measure accuracy on the holdout fold
 - Choose tree with best performance
 - Assess its performance on the validation, i.e., test data, not used at all to this point

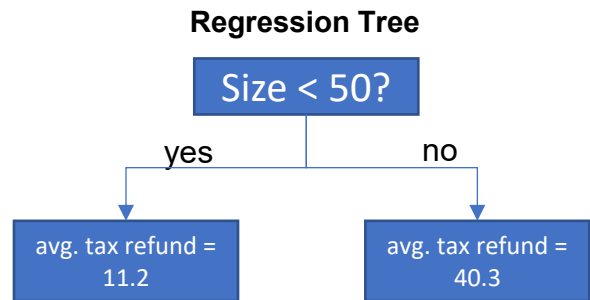
Regression Trees

Regression Trees

- The same as classification trees, but when the output/outcome (the target we are trying to predict) is numerical rather than categorical
- Example: predicting the “price” of used cars, the “price” of houses, the “tax refund amount”, etc.
- In every result “box” rather, than list the number of events “true, 1” vs “false, 0”, we instead compute the “average” value



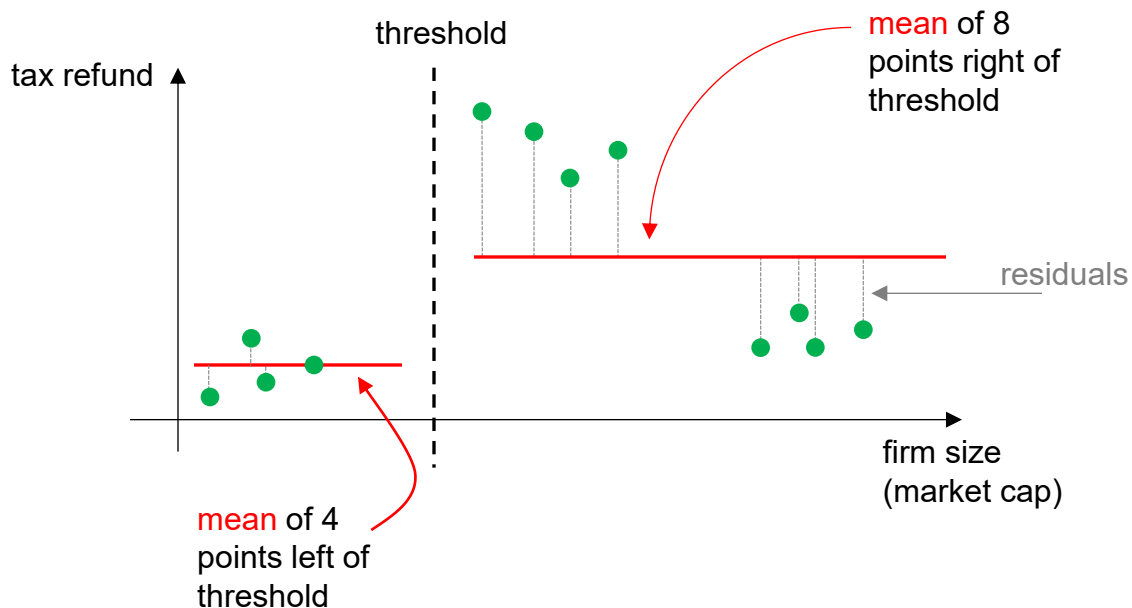
uses misclassification, Gini or entropy to pick a split



uses instead Sum of Squared residuals/errors (SSR) from mean

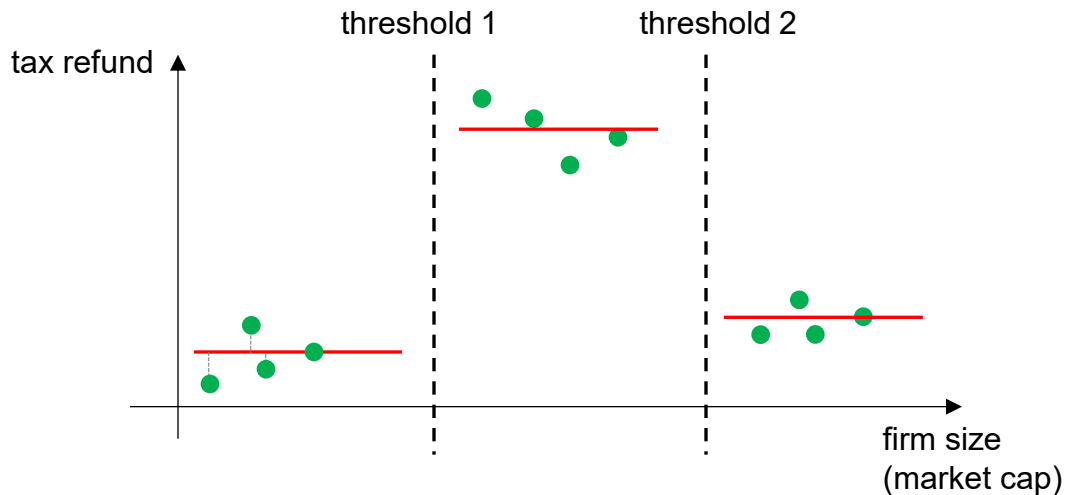
Regression Trees - SSR

- At each box, for every possible threshold... add up the squared residuals



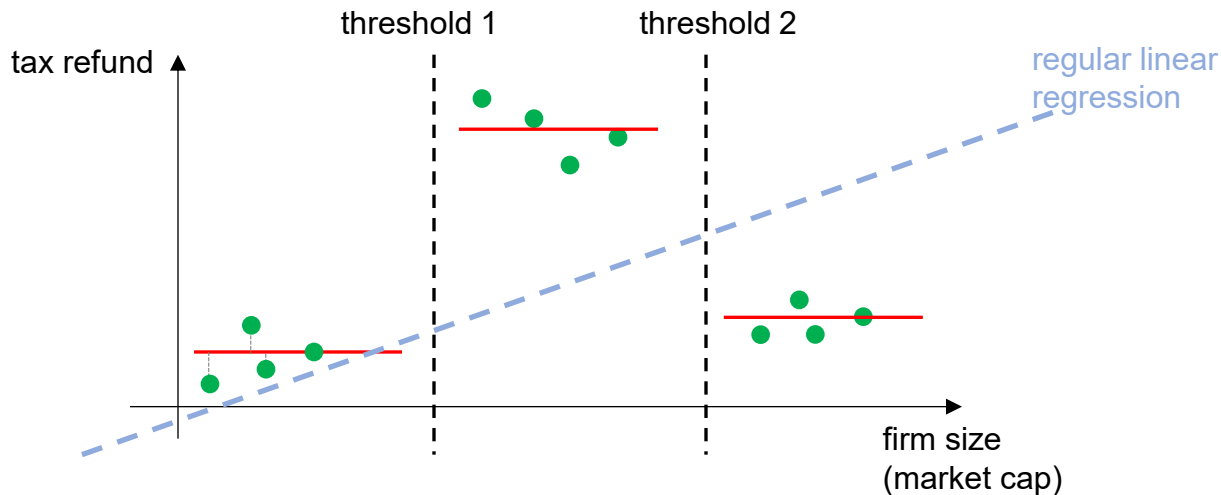
Regression Trees - SSR

- The tree will (eventually) find the right splits



Regression Trees vs Linear Regression

- For this type of data, linear regression underperforms (assuming right thresholds are chosen)

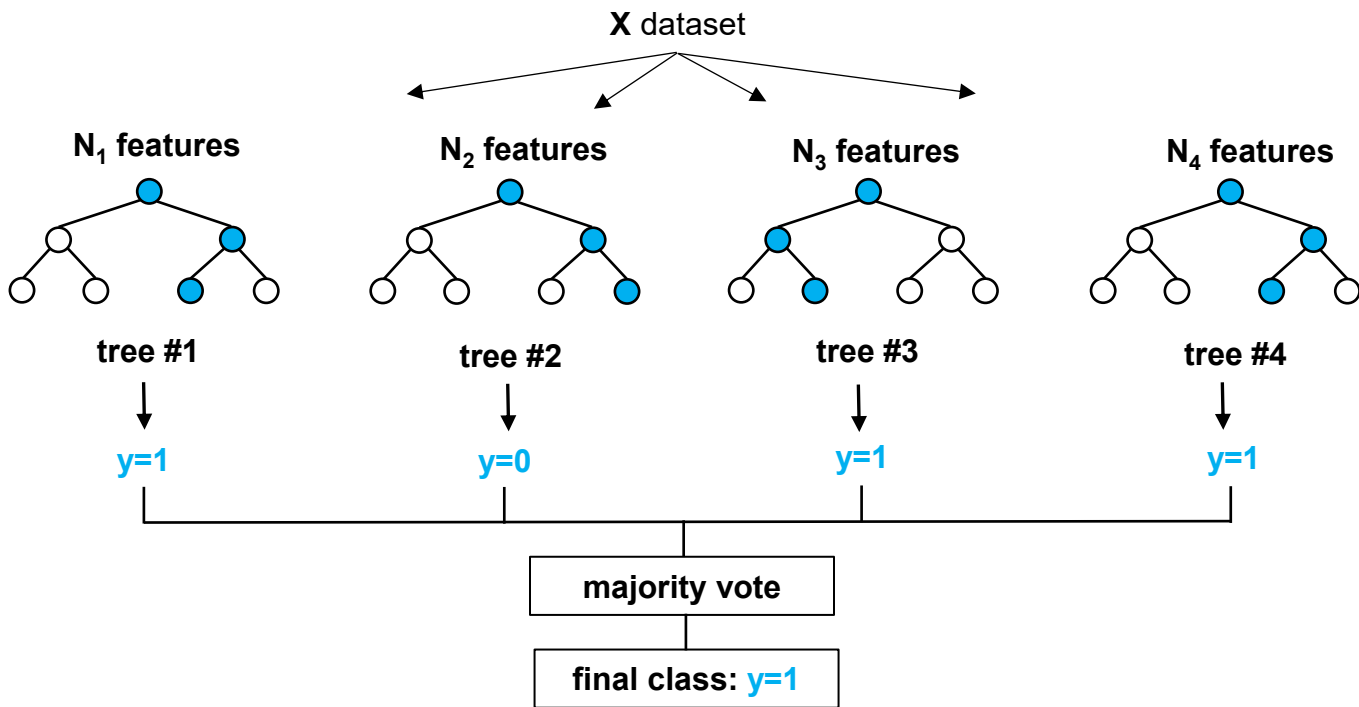


Random Forests

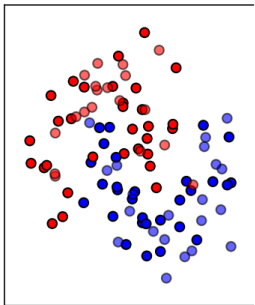
Making Trees Better – Random Forest

- A decision tree is ideal when the rules are critical (e.g., medical settings)
- However, if we are primarily concerned with prediction accuracy, better performance is provided by several extensions to trees that combine results from multiple trees. These are called **ensemble** methods.
- One popular method is called “**Random Forest**”
 - 1. draw random samples from the dataset, with replacement (this sampling approach is called the **bootstrap**).
 - 2. randomly choose a subset of predictors (columns) and run a classification (or regression) tree
 - 3. repeat steps 1-2 multiple times (~500 times), thus obtaining a “forest” (collection of trees)
 - 4. combine the predictions/ classifications from the individual trees to obtain improved predictions.
 - use “majority voting” for classification or simple “averaging” for numerical prediction

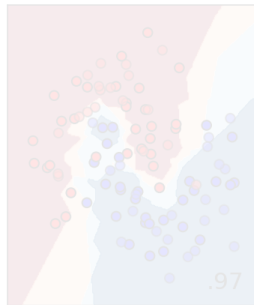
Making Trees Better – Random Forest



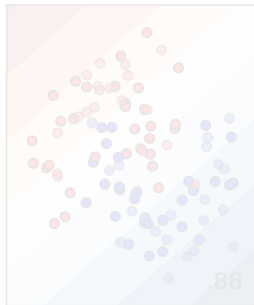
Input data



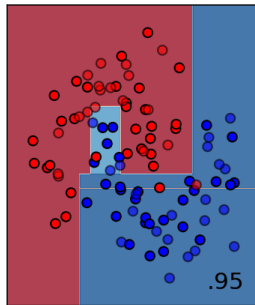
Nearest Neighbors



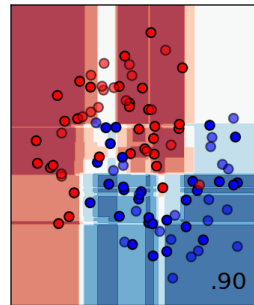
Linear SVM



Decision Tree



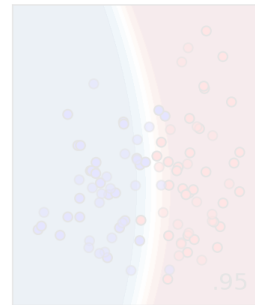
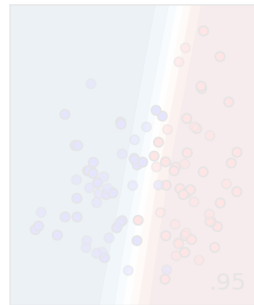
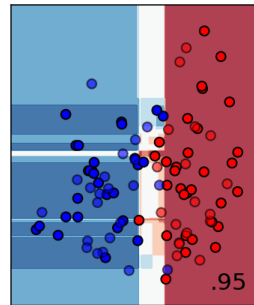
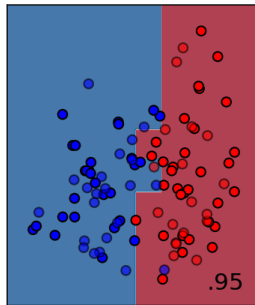
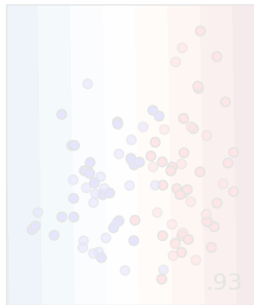
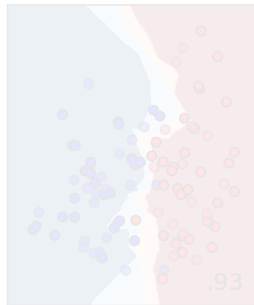
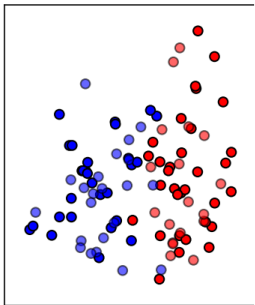
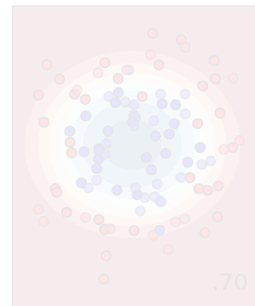
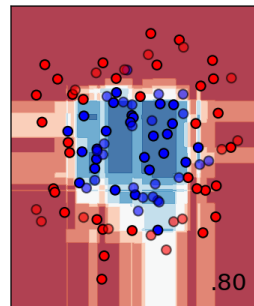
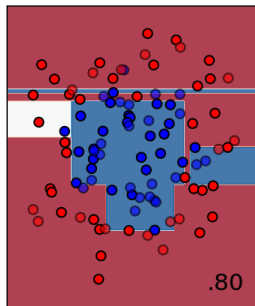
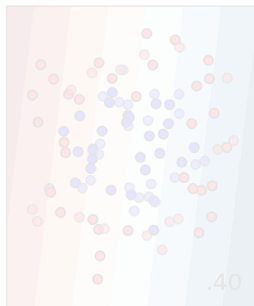
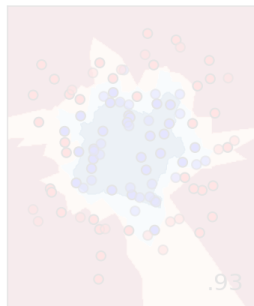
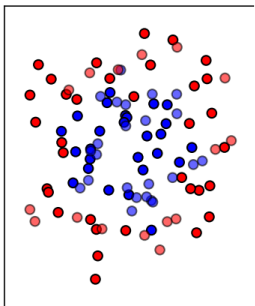
Random Forest



Neural Net



Naive Bayes



Summary

- Classification and Regression Trees are an understandable and transparent method for classifying or predicting new records
- A single tree is a graphical representation of a set of rules
- Tree growth must be stopped to avoid overfitting of the training data
- Ensembles (random forests) improve predictive performance, but you lose interpretability and the rules embodied in a single tree
- Advantages
 - Easy to use, understand
 - Single trees produce rules that are easy to interpret & implement
 - Variable selection & reduction is automatic
 - Do not require the assumptions of statistical models
 - Can work without extensive handling of missing data
- Disadvantage of single trees
 - Instability and poor predictive performance